# Modeling languages: alpha and omega

João Pedro PEDROSO|DIONÍSIO

Faculty of Sciences, University of Porto

2026 APDIO Workshop: Optimization Software & Practice
University of Minho, 27-MAR-2026

# Outline


1. The "Alpha" of Optimization
2. Model taxonomy
3. The "good" modeling language checklist
4. Modern usage: the "Omega" of Optimization
5. Real-world deployment

# Background

- We have a system of linear inequalities.
- How can we find a feasible solution?
- In particular, can we choose a feasible solution that optimizes some (linear) criterion?
  - *note: that amounts to optimizing an expression, which can be assigned to a variable.*

# The Pre-History: Joseph Fourier (1826)

## The Concept of Projection

Before the Simplex Method: Fourier devised a method for solving systems of linear inequalities:

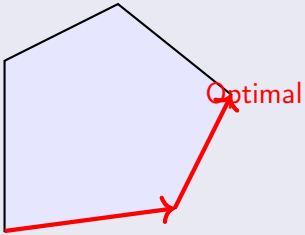
- Method: Fourier–Motzkin Elimination.
- Goal: Determining the feasibility of undetermined systems.
- Insight: Optimization (min-max) is equivalent to solving inequalities.

# Comparing "Projecting" vs. "Walking"

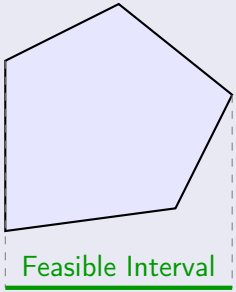
- Fourier (1826):
  - Project the $n$-dimensional feasible region onto lower dimensions.
- Dantzig (1947):
  - *Simplex Method:* Move between vertices of the $n$-dimensional region.

# Visualizing the Two Approaches

## Vertex Walk vs. Projection




Simplex: Vertex Walk




Fourier: Projection

# Fourier's Method "Explodes"

- Fourier-Motzkin: Each variable elimination creates a new set of inequalities.
  - If we have $m$ inequalities, eliminating one variable can result in up to $(m/2)^2$ new ones.
  - Impractical for the next example (Stigler's 77 variables).
  - Theoretically important (e.g., for proving the Farkas Lemma).
- Simplex: Complexity depends on the number of vertices (usually efficient).

**Historical Note**

Dantzig noted that had he known Fourier–Motzkin Elimination he might have been discouraged by the perceived complexity of inequality systems.

# The Stigler Diet Problem (1947)

## A Problem in the "Alpha" Era

George Stigler (1945) sought the minimum cost to feed a 154lb man.

- Goal: Minimize daily cost of food.
- Scale: 77 candidate foods, 9 nutritional constraints.

## Mathematical Formulation

- Minimize $Z = \sum_{j=1}^{77} c_j x_j$
- Subject to: $\sum_{j=1}^{77} a_{ij} x_j \geq b_i \quad \forall i \in \{1, \ldots, 9\}, \qquad x_j \geq 0 \quad \forall j \in \{1, \ldots, 77\}$
- Where:
  - $c_j$: Cost of food $j$
  - $a_{ij}$: Amount of nutrient $i$ in food $j$
  - $b_i$: recommended daily allowance for nutrient $i$
    calories, protein, calcium, iron, vitamin A, B1, B2, niacin, vitamin C

# The Human "Computer": Jack Laderman's Achievement

## Before Machines

In 1947, the "Simplex Method" was brand new (Dantzig). There was no "software."

- Team: 9 human clerks at the National Bureau of Standards.
- Hardware: Hand-operated desk calculators
- Effort: 120 man-days of computation.

## Results

- Stigler's heuristic guess: $39.93/year (1939 prices).
- Laderman's Simplex solution: $39.69/year.
- Fun Fact: The "optimal" diet consisted mostly of wheat flour, evaporated milk, cabbage, spinach, and navy beans. Stigler called it the "human dog biscuit."

# First Machine Implementations: SEAC and IBM 701

## SEAC (1951) - The First Electronic Solver

- Machine: Standards Eastern Automatic Computer (National Bureau of Standards).
- Implementation: First general-purpose Simplex code for a stored-program computer.
- Language: Machine Code / Binary (no high-level languages like Fortran existed yet).
- Scale: Solved a $48 \times 71$ Air Force model in minutes.

## IBM 701 (1954) - The First "Commercial" Code

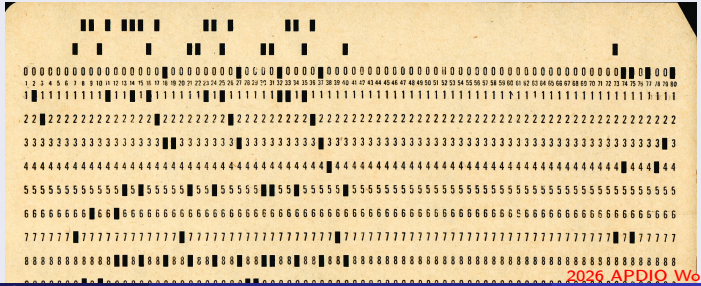
- Developed at RAND Corporation.
- Language: Assembly (highly optimized for the 701's architecture).
- Could handle ~100 constraints
  - moved LP from research to the oil industry (refining/blending).
- Beginning of Matrix Generators
  - (tedious) precursors to the modeling languages we use today.

# Before Modeling Languages: Physical Data Entry

## The Punched Card Era (1950s)

In the "Alpha" phase, there was no `model.lp` or `model.py`.
Data was physical.

- Storage: IBM 80-column punched cards.
- Method: One non-zero coefficient per card.
- Hardware: IBM 026 Keypunch and 056 Verifier.



# Before Modeling Languages: Workflow

1. Mathematical Model: written on paper.
2. Matrix Tabulation: coefficients mapped to row/column indices.
3. Keypunching: manual transcription to cards.
4. Verification: mechanical "double-checking" of holes.

## Technical Bottleneck

- A typo in one card meant the Simplex algorithm would diverge or find a wrong solution.
- There was no "algebraic" check, only a mechanical one.
- Transition: creation of Matrix Generators in the 1960s.

# The "Middle Ages": Matrix Generators

## The "Logic" of 1965

- Instead of writing math, we wrote code to print strings and formatted numbers.
  - specialized "printing" programs usually written in FORTRAN.
- Any mistake in the code makes the MPS file unreadable/wrong.
- Maintenance: To add a nutrient, we must change the ‘DATA‘ statements, the ‘DO‘ loop limits, and potentially the ‘FORMAT‘ strings.

# The "Middle Ages"

## A Matrix Generator (Fortran)

```
C     FORTRAN MATRIX GENERATOR FOR STIGLER DIET
      DATA FOODS/'WHEAT', 'CORNM', 'EVAPM', 'MARGA'/
      DATA NUTRI/'COST ', 'CALOR', 'PROT ', 'CALCI'/

      WRITE(6, 100) 'NAME', 'STIGLER'
      WRITE(6, 110) 'ROWS'
      WRITE(6, 120) ' N ', 'COST'
      DO 10 I=2, 5
   10 WRITE(6, 120) ' G ', NUTRI(I)

      WRITE(6, 110) 'COLUMNS'
      DO 20 J=1, 77
C       INNER LOOP FETCHES DATA FROM ARRAYS AND FORMATS FOR MPS
        WRITE(6, 130) FOODS(J), 'COST', COST(J), 'CALOR', CAL(J)
   20 CONTINUE

  100 FORMAT(A4, 10X, A8)
  110 FORMAT(A)
  120 FORMAT(A3, 2X, A8)
  130 FORMAT(4X, A8, 2X, A8, F12.4, 2X, A8, F12.4)
```

# The MPS Format (The Lingua Franca)

Standardized by IBM in 1965, it remains the "Assembly Language" of LP.

- Rigid Structure: 80-column fixed-field layout (inherited from punched cards).
- Sections: `NAME`, `ROWS`, `COLUMNS`, `RHS`, `BOUNDS`.
- Persistence: Every modern solver (Gurobi, CPLEX, HiGHS) still reads .mps files.

## MPS Paradox

- Highly efficient for computers to parse.
- Nearly impossible for humans to debug.
- This "Readability Gap" is what eventually forced the creation of AMLs.

# MPS Format

## Example: Stigler Diet in 80 Columns

```
NAME          STIGLER
ROWS
 N  COST
 G  CALORIES
 G  PROTEIN
 G  CALCIUM
COLUMNS
    WHEAT     COST          0.036     CALORIES      3.3
    WHEAT     PROTEIN       13.3      CALCIUM       0.9
    CORNMEAL  COST          0.018     CALORIES      3.0
    CORNMEAL  PROTEIN       10.0      CALCIUM       2.0
RHS
    RHS1      CALORIES      3.0       PROTEIN       70.0
    RHS1      CALCALCIUM    0.8
ENDATA
```

# The Representation Gap

## From Human Thought to Solver Input

- Before AMLs (like AMPL or JuMP), there was a "lost in translation" phase.
  - To a mathematician, the model is a set of summations.
  - To the computer, the model is a flat list of entries.

## Comparison: The Stigler Diet Problem

| Format | Representation of $\sum a_{ij}x_j \geq b_i$ | Readability |
|---|---|---|
| Matrix Generator | `FOR I=1 TO 9: WRITE(6,10) ROW(I), RHS(I)` | Zero |
| MPS (Machine) | `COL001 NUTRIENT01 0.0034` | |
| Algebraic | $\sum_{j \in J} a_{ij}x_j \geq b_i, \quad \forall i \in I$ | High |

# MPS: "Technical Debt" of the 1970s

- In MPS, data and logic are fused:
  - data ($a_{ij}$) is separate from logic (a sum).
  - if we add one nutrient to the diet problem, we must recalculate and re-punch the entire file.
- Modeling was slow, error-prone, and required "model librarians".
- Support for linear programming, with limited extensions added:
  - 1965: original MPS, linear programming.
  - 1969: integer variables → `INTORG` and `INTEND` markers within the `COLUMNS` section.
  - 1970: limits on the variables → `BOUNDS` section
  - 1976: piecewise linear functions → Special Ordered Sets:
    - `SOS1`: Only one variable in the set can be non-zero.
    - `SOS2`: At most two variables can be non-zero, and they must be adjacent (used for approximating curves).
  - 1980s: $x^2$ in the objective: `QCMATRIX` or `QUADOBJ`
- but format still rather limited...

# Model Taxonomy: Choosing the Right Solver

## The "Linear" Foundation

- Linear Programming (LP): Continuous variables, linear constraints and objective.
  - Standard, typically uses simplex or interior point methods.
- Mixed-Integer Programming (MIP)*: Adds discrete choices (0/1).
  - NP-Hard problems, typically solved with Branch-and-Bound.

## Beyond Linearity

- Nonlinear Programming (NLP): Curved surfaces (e.g., chemical blending, power grids).
  - Local vs. Global optima, typically requires gradient-based solvers.
- Constraint Programming (CP): Logic constraints (e.g., "If A is before B, then C cannot be at 4pm").
  - Applications: Logic programming, Scheduling, Routing.
- Complementarity problems: "Either $x = 0$ or the constraint is tight."
  - Applications: Game theory, equilibrium models.

# The Renaissance: GAMS (1976)

## Algebraic Realism

In GAMS, the model finally looks like the math. Note the separation of Sets, Data, and Equations.

```
Sets
   i   nutrients  / calories, protein, calcium /
   j   foods      / wheat, cornmeal, milk / ;

Table a(i,j)  nutritional value
              wheat  cornmeal  milk
   calories    3.3     3.0      0.5
   protein    13.3    10.0      1.0 ;

Variables  x(j)  purchases,  cost ;
Positive Variable x;

Equations  nbb(i)  nutrient balance,  obj  total cost ;

nbb(i) ..  sum(j, a(i,j) * x(j)) =g= b(i) ;
obj ..     cost =e= sum(j, c(j) * x(j)) ;
```

# The "good" modeling language checklist

1. Decoupling data from the model logic
   - we can change the data and resolve, without touching the model
2. Solver independence
3. Declarative (vs. Imperative)
   - tell what the problem is, not how to find a solution
4. Readability
5. Handling:
   - presolve phase
   - indexing
   - sparsity
   - solver input/output

# Solver Independence

- We write the model in a human-readable form (mathematical program).
- The AML translates it into the specific binary format required by the solver
  - CPLEX
  - IPOPT
  - Gurobi
  - SCIP
  - ...
- Benefit: We can switch solvers with one command.
  - e.g., `option solver = scip;`.

# Modern usage

- Dedicated languages (AMPL, GAMS, . . . ) vs.
- Embedded libraries (Pyomo, JuMP, GurobiPy, . . . )

## Dedicated AMLs (AMPL, GAMS)

- Nature: Domain-specific languages (DSL).
- Strengths:
  - Syntax is mostly identical to mathematical notation.
  - High-performance presolving and model generation.
  - Support for a vast range of solvers.
- Weakness:
  - Hard to connect to databases, web APIs, or ML libraries (e.g., scikit-learn). (If used as a standalone system)

## Embedded Libraries (Pyomo, JuMP, GurobiPy)

- Nature: Optimization modeling *inside* a general-purpose language.
- Strength: Total integration. The model is just an object in a Python/Julia script.
  - Pull data from a SQL database, solve, and push results to a Dashboard in one script.
- Weakness: "Language Overhead"
  - Python can be slower than a dedicated AML for massive model generation.
  - Limited range of solvers supported.

# Syntax Showdown: AMPL vs. Pyomo

## The Objective Function: $\sum_j c_j x_j$

**AMPL (Dedicated)**

```
var x {FOODS} >= 0;
minimize Cost:
    sum {j in FOODS} c[j] * x[j];
```

**Pyomo (Embedded)**

```
model.x = Var(model.FOODS, domain=NonNegativeReals)
def obj_rule(model):
    return sum(c[j] * model.x[j] for j in model.FOODS
model.obj = Objective(rule=obj_rule)
```

- AMPL is shorter and closer to paper math.
- Pyomo is more "verbose" but allows fully using Python language, debuggers, and libraries.

# Modern Usage

## The Best of Both Worlds

The modern landscape is no longer a binary choice between "Dedicated" and "Embedded."

- The Hybrid Approach: Using a DSL (AMPL/GAMS) inside a GPL (Python/Julia).
- The Interface: Libraries like `amplpy` or `GAMS Transfer`.
- Advantages:
  1. Modeling: The math remains clean, concise, and separate from the "plumbing."
  2. Execution: The AMPL engine handles high-speed model generation (C++ core).
  3. Integration: E.g., data comes from Pandas/SQL; results go to Plotly/Dash.

## Competitive Advantage

- Readability: The model looks like a paper (AMPL syntax).
- Scalability: Avoids Python's "Global Interpreter Lock" (GIL) during model build.
- Deployment: Easy to containerize (Docker) while keeping the core logic in a DSL.

# The Unified Syntax: amplpy

## From Data Science to Optimization

Instead of "translating" math into Python loops, we pass the data into an AMPL environment.

```
from amplpy import AMPL
import pandas as pd

# 1. The "Alpha": Pure Algebraic Model
ampl = AMPL()
ampl.eval("""
    set FOODS;
    param cost{FOODS};
    var x{FOODS} >= 0;
    minimize TotalCost: sum{j in FOODS} cost[j] * x[j];
""")

# 2. The "Modern": Python Data Integration
df = pd.read_sql("SELECT name, price FROM market_data", db_conn)
ampl.set_data(df, "FOODS")

# 3. The "Solver": Solver Independence
ampl.set_option('solver', 'gurobi')
```

# Modern Tech Stack: A Modeler's Compass

## The Selection Criteria

Choice depends on Build Speed, Model Complexity, and Production Ecosystem.

| Tool | Category | Language | Best For... |
|---|---|---|---|
| GAMS | Standalone DSL | GAMS | Massive-scale NLP/Economics; legacy reliability. |
| AMPL | Standalone DSL | AMPL | Mathematical purity; extremely fast model generation. |
| Pyomo | Embedded | Python | Academic research; complex custom algorithms. |
| PuLP | Wrapper | Python | Beginners; quick prototypes; simple Linear models. |
| JuMP | Embedded | Julia | Performance-critical research; "DSL speed" in a GPL. |
| amplpy | Unified API | Python + AMPL | The Industrial Standard: DSL speed + Python power. |

- If Model Generation Time is the bottleneck (massive matrices) $\rightarrow$ AMPL/amplpy or JuMP.
- If Ease of Integration with a Flask/Django web app is the goal $\rightarrow$ PuLP or amplpy.
- If solving Non-linear Global Optima $\rightarrow$ GAMS and AMPL are best for solver support.

# The Evolution of the Modeling Interface

## From Physical Cards to Unified APIs

The "Omega" is not just about syntax; it is about how the Model communicates with the Data and the Solver.

| Era | Interface Method | Primary Pain Point |
|---|---|---|
| Middle Ages | Punched Cards / MPS | Manual typos and physical loss |
| Renaissance | Matrix Gen $\rightarrow$ MPS | Rigid "File-based" I/O; logic buried in Fortran |
| Modern (early) | Modeling languages (GAMS, AMPL, ...) | Integration in the deployed system |
| Modern (middle) | Pure Python (Pyomo) | "Language Overhead" (slower model generation) |
| Unified Approach | API-driven DSL (amplpy) | Licensing and environment complexity |

## Why "Unified Approach" Wins

- The math (`.mod`) is separate from the data (`.csv/SQL/...`).
- Speed: The DSL (AMPL) generates the model in optimized C++ routines.
- Python handles the "Pre" and "Post" processing (Visualization, ML).
- We can stop, modify, and re-solve without writing a single file to disk.

# Advanced Topics: Column Generation

## The "Orchestrator" Pattern

Column generation requires solving a Master Problem (MP) and a Subproblem (SP) iteratively.

```
# 1. Initialize Master and Subproblem in AMPL
master = AMPL(); master.read("master.mod")
sub    = AMPL(); sub.read("subproblem.mod")

while True:
    master.solve()
    # Extract duals from Master using Python API
    pi = master.get_con('demand_constraints').dual()

    # Pass duals as parameters to Subproblem
    sub.get_parameter('pi').set_values(pi)
    sub.solve()

    if sub.get_objective('ReducedCost').value() >= -1e-6:
        break  # Optimality reached

    # Generate new column (variable) in Master
    new_column = sub.get_variable('x_new').get_values()
```

# Advanced Topics: Cutting Planes

## Bridging Heuristics and Exact Math

In modern usage, "Cuts" are often generated by analyzing the fractional solution of an LP relaxation.

- The Workflow:
  1. Solve the Relaxed Problem (ignore integrality).
  2. If the solution is fractional, use a Separation Algorithm (in Python).
  3. Identify a "Violated Constraint" (e.g., a Gomory cut or Cover cut).
  4. Inject that constraint back into the live AMPL object.
  5. Repeat until the solution is integer or no more cuts are found.

## Advantage of the API

- No need to restart the solver from scratch.
- The solver (Gurobi/CPLEX/...) performs a warm start, keeping the previous basis.
- This capability is only possible via memory-resident APIs (`amplpy`), not file-based MPS.

# Real-World Deployment: Beyond the Notebook

## The Modern Infrastructure

In the 1950s, the "Interface" was a card reader. Today, it is a Distributed Cloud.

- Cloud Solvers: Instead of a local license, the API sends the model to a high-performance cluster (Gurobi Cloud, NEOS, Azure/AWS).
  - Advantage: Massive parallelization for MIP or Stochastic models.
- Model-as-a-Service (MaaS):
  - The model is wrapped in a REST API (FastAPI/Flask).
  - The UI (Frontend) sends data $\rightarrow$ API solves $\rightarrow$ UI displays results.

## Integration with Data Science Pipelines

- The Workflow:
  1. Pre-processing: Pandas/SQL clean the "noisy" real-world data.
  2. Predictive: ML models (Scikit-learn/XGBoost) predict parameters (e.g., next week's demand).
  3. Prescriptive: The Unified Omega (AMPL/Python) solves for the optimal plan based on those predictions.

# Conclusion: From Alpha to Omega

## The Journey of Abstraction

- The Alpha: Mathematical truth remains constant (Dantzig's Simplex).
- The Middle Ages: Physical struggle (Punched cards, Matrix Generators).
- The Renaissance: Separation of concerns (GAMS/AMPL, Decoupling).
- The Omega: The Unified API (DSL + GPL).

## Takeaways for the Modern Modeler

1. Choose the right taxonomy: Don't use a MIP solver for a nonlinear problem.
2. Decouple everything: Keep math (`.mod`) separate from the data (`.csv/...`).
3. Automate the iteration: Use APIs (`amplpy`) for Column Gen or Cutting Planes.
4. Deploy with purpose: A model is only useful if it integrates into the "Decision Pipeline."

# Further Reading and References: The "Alpha" (Foundational Papers)

- Stigler, G. J. (1945): "The Cost of Subsistence." Journal of Farm Economics. (The Diet Problem).
- Dantzig, G. B. (1947): "Maximization of a Linear Function of Variables Subject to Linear Inequalities." (The Simplex Method).
- Benders, J. F. (1962): "Partitioning procedures for solving mixed-variables programming problems." Numerische Mathematik.

# Further Reading and References: The "Middle Ages" and "Renaissance"


- Orchard-Hays, W. (1968): Advanced Linear-Programming Computing Techniques. (The MPS Era).
- Bisschop and Meeraus (1982): "On the Development of a General Algebraic Modeling System in a Strategic Planning Environment." (Birth of GAMS).
- Dantzig, G. B. (1985): Impact of linear programming on computer development. Computers in Mathematics, 233–240. https://doi.org/10.1201/9781003072157-6
- Fourer, Gay, and Kernighan (1990): "A Modeling Language for Mathematical Programming." (Birth of AMPL).
- Gass, S. I. (2002): The first linear-programming shoppe. Operations Research, 50(1), 61–68. https://doi.org/10.1287/opre.50.1.61.17781
- Bixby, R. E. (2012): A brief history of linear and mixed-integer programming computation. Documenta Mathematica Series, 107–121. https://doi.org/10.4171/dms/6/16

# The "Omega": Modern APIs and Libraries

- Pyomo Documentation: `http://www.pyomo.org`
- JuMP (Julia) Portal: `https://jump.dev`
- Gurobi/CPLEX Python Guides: Official developer portals for callback implementation.
- AMPL Python API: `https://amplpy.ampl.com`